Practical 2: State Management and Routing in React

Problem Statement:

• Extend the portfolio application from Practical 1 by adding React Router.

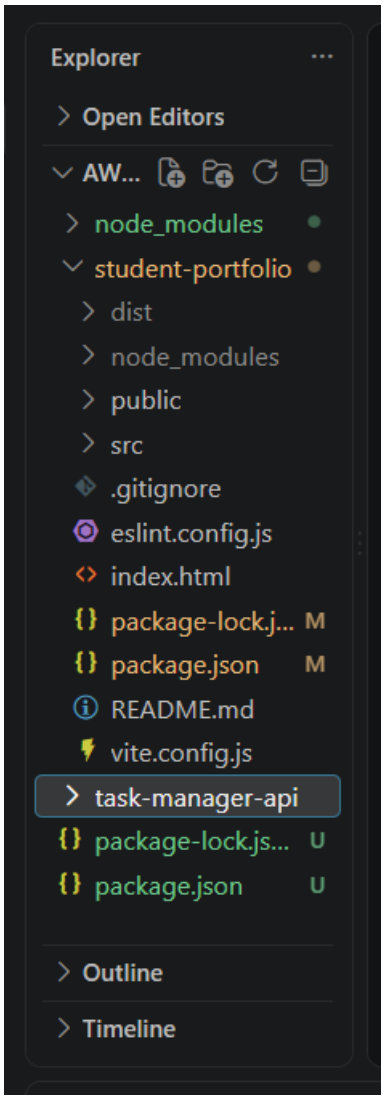
• Implement a navigation bar with at least 3 routes: Home, Projects, and Contact.

• Each route must render a distinct component without a full page reload.

• Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page.

• The Contact page must include a controlled input that captures and displays user input in real time.

Folder Structure:



App.jsx

```
import { useState } from "react";
import { Routes, Route } from "react-router-dom";
import "./App.css";

import NavBar from "./components/NavBar";

import Home from "./Pages/Home";
import Project from "./Pages/Projects"; // Make sure the file is Projects.jsx
import Contact from "./Pages/Contact";
import NotFound from "./Pages/NotFound";

function App() {
  const [darkMode, setDarkMode] = useState(false);

  return (
    <div className={darkMode ? "dark app" : "light app"}>
      <NavBar />

      <div className="theme-btn">
        <button onClick={() => setDarkMode(!darkMode)}>
          {darkMode ? "☀ Light Mode" : " ◗ Dark Mode"}
        </button>
      </div>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/projects" element={<Project />} />
```

```
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
  );
}

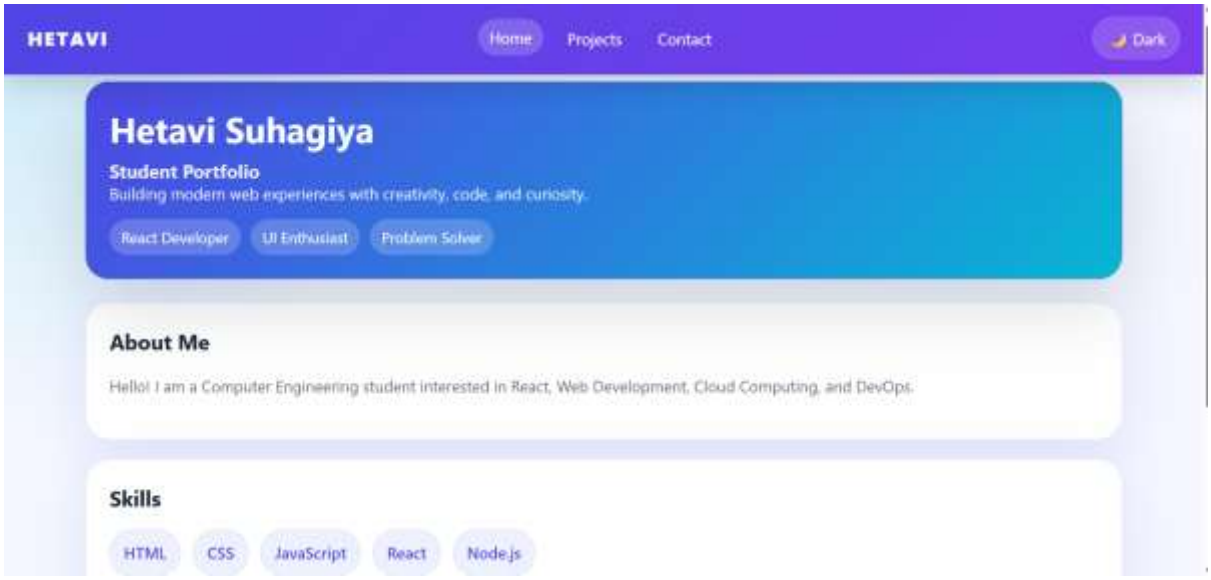
export default App;
```

Home.jsx

```
import Header from "../components/Header";
import About from "../components/About";
import Skill from "../components/Skill";
import Footer from "../components/Footer";

function Home() {
  return (
    <>
      <Header />
      <About />
      <Skill />
      <Footer />
    </>
  );
}

export default Home;
```



Projects.jsx

```
import { useEffect, useState } from "react";
import Spinner from "../components/Spinner";
import ErrorMessage from "../components/ErrorMessage";

function Projects() {
  const [repos, setRepos] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [search, setSearch] = useState("");

  const fetchRepos = async () => {
    setLoading(true);
    setError(null);

    try {
      const response = await fetch(
        "https://api.github.com/users/datri2317/repos"
      );
```

```
    if (!response.ok) {
      throw new Error("Failed to fetch repositories.");
    }

    const data = await response.json();
    setRepos(data);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchRepos();
}, []);

const filteredRepos = repos.filter((repo) =>
  repo.name.toLowerCase().includes(search.toLowerCase())
);

if (loading) return <Spinner />;

if (error)
  return (
    <ErrorMessage
      message={error}
      onRetry={fetchRepos}
    />
  );
```

```
 return (
   <div className="container">
    <h1>My GitHub Projects</h1>

    <input
     type="text"
     placeholder="Search Repository..."
     value={search}
     onChange={(e) => setSearch(e.target.value)}
    />

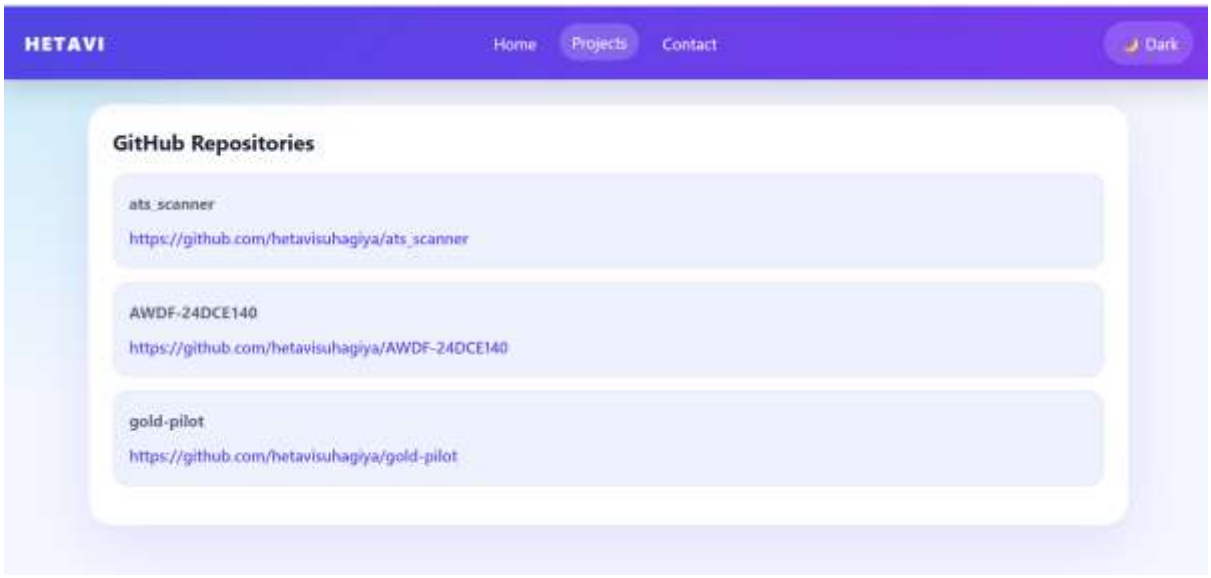
    <div className="repo-grid">
     {filteredRepos.map((repo) => (
      <div className="repo-card" key={repo.id}>
       <h2>{repo.name}</h2>

       <p>⭐ Stars : {repo.stargazers_count}</p>

       <a
        href={repo.html_url}
        target="_blank"
        rel="noreferrer"
       >
        View Repository
       </a>
      </div>
     ))}
    </div>
   </div>
```

```
  );
}

export default Projects;
```



Contacts.jsx

```
import { useState } from "react";

function Contact() {

  const [message, setMessage] = useState("");
  const [showHelp, setShowHelp] = useState(false);

  return (
    <div className="container">

      <h1>Contact Me</h1>

      <input
        type="text"
        placeholder="Enter your message..."
```

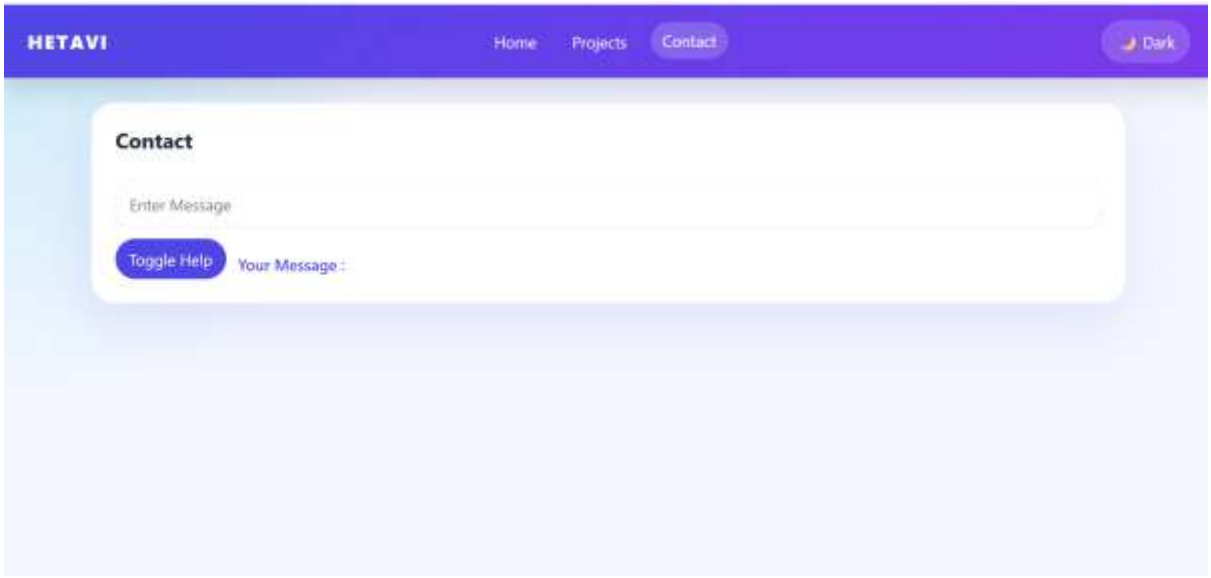
```
        value={message}
        onChange={(e) => setMessage(e.target.value)}
      />

      <p>Character Count : {message.length}</p>

      <button onClick={() => setShowHelp(!showHelp)}>
        {showHelp ? "Hide Help" : "Show Help"}
      </button>

      {showHelp && (
        <div className="help">
          Enter your message and submit it.
        </div>
      )}

    </div>
  );
}
export default Contact;
```



Key Questions:

• What is the role of each component in the overall UI structure?

Each component is responsible for a specific part of the user interface. For example, the Header displays the top section, the Home component displays the main page, the Project component displays project details, and the Footer displays the bottom section. This makes the UI easier to manage and organize.

• How does React re-render when props change?

When the props of a component change, React detects the change and re-renders that component. It updates the UI with the new data received from the parent component.

• Why is component reusability important in large-scale applications?

Component reusability allows us to use the same component in different parts of an application without writing the same code again. It reduces code duplication, saves time, and makes the application easier to maintain and manage.

Supplementary Problems:

• Add a 404 Not Found route that renders a custom error component for unknown paths.

• Implement a dark/light mode toggle using useState that applies a CSS class to the root element.

• Store the contact form input in state and display a live character count below the input.

NotFound.jsx

```
import { Link } from "react-router-dom";


function NotFound() {
  return (
    <div className="notfound">

      <h1>404</h1>

      <h2>Page Not Found</h2>

      <p>
```

```
      Sorry, the page you are looking for does not exist.
    </p>

    <Link to="/">
      <button>Go Home</button>
    </Link>

  </div>
 );
}

export default NotFound;
```

Light/Dark Mode:

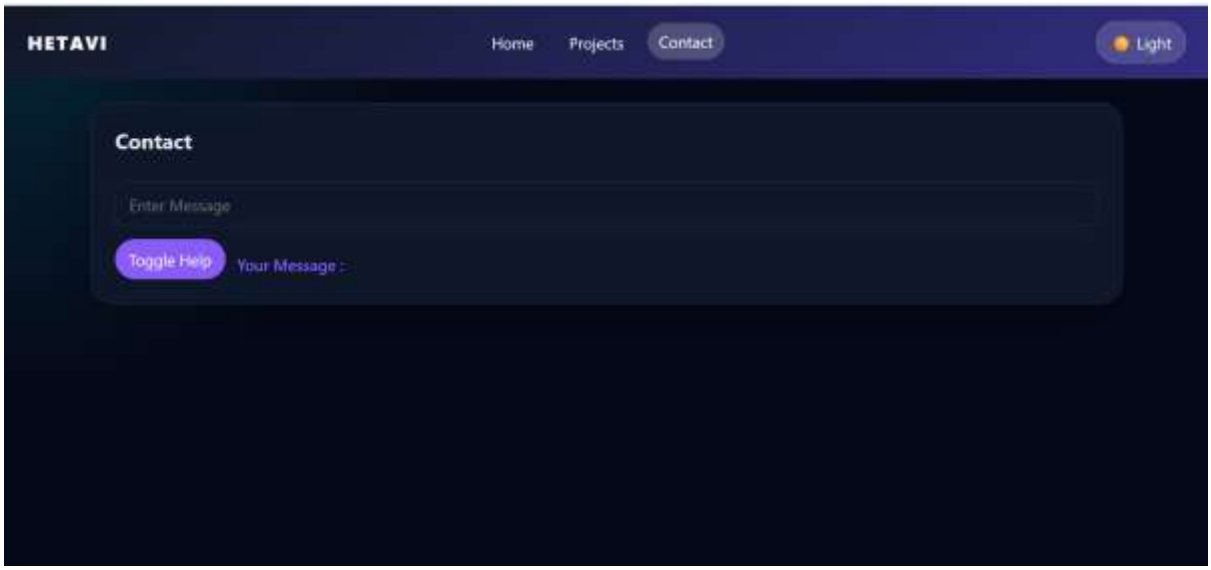
```
function App() {
 const [darkMode, setDarkMode] = useState(false);

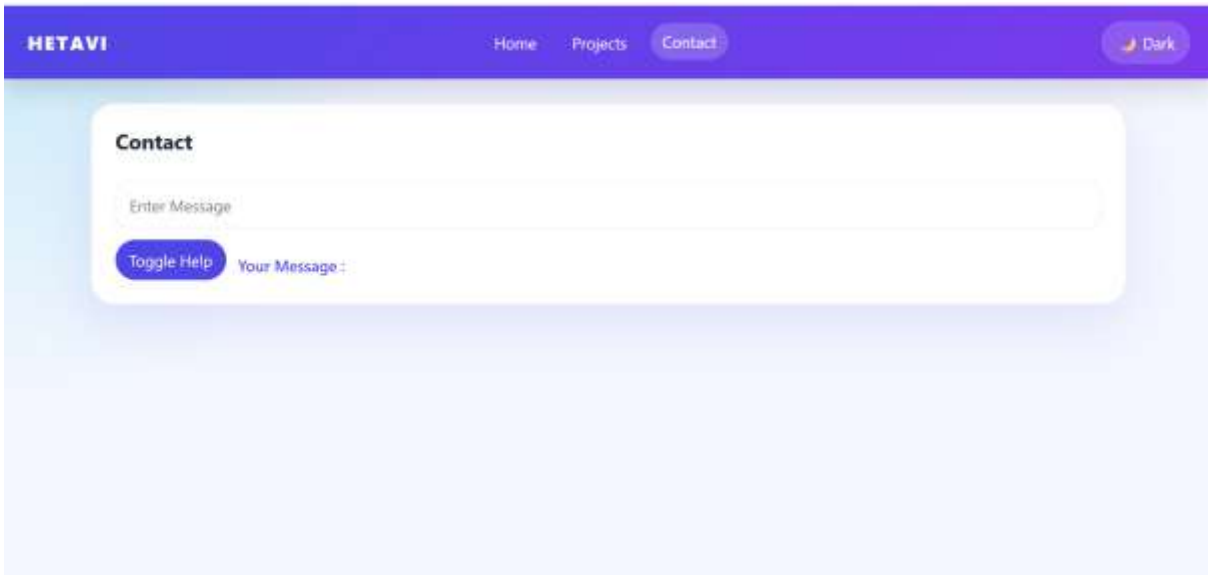
 return (
  <div className={darkMode ? "dark app" : "light app"}>
    <NavBar />

    <div className="theme-btn">
      <button onClick={() => setDarkMode(!darkMode)}>
        {darkMode ? "☀ Light Mode" : " ☽ Dark Mode"}
      </button>
    </div>

    <Routes>
      <Route path="/" element={<Home />} />
```

```
        <Route path="/projects" element={<Project />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
  );
}
```





Contacts.jsx

```
import { useState } from "react";

function Contact() {
```

```
 const [message, setMessage] = useState("");
 const [showHelp, setShowHelp] = useState(false);

 return (
  <div className="container">

    <h1>Contact Me</h1>

    <input
      type="text"
      placeholder="Enter your message..."
      value={message}
      onChange={(e) => setMessage(e.target.value)}
    />

    <p>Character Count : {message.length}</p>

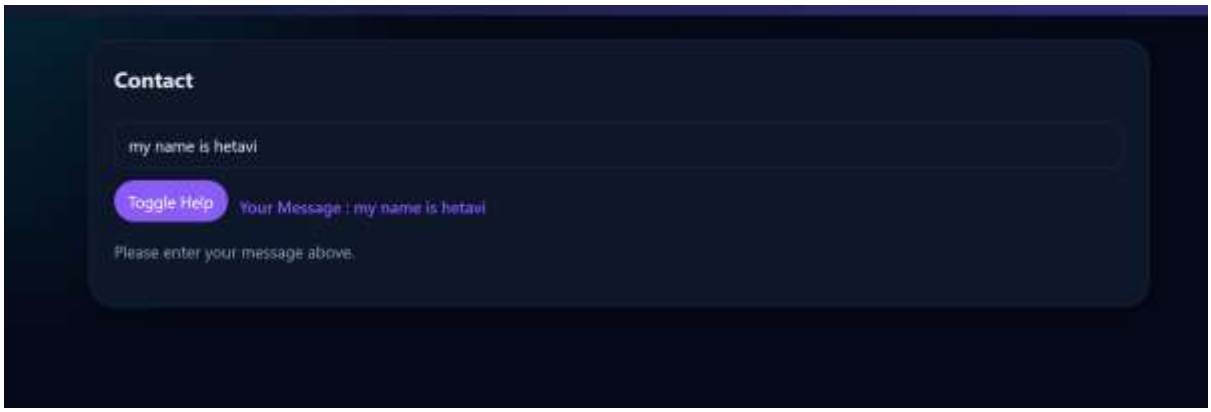
    <button onClick={() => setShowHelp(!showHelp)}>
      {showHelp ? "Hide Help" : "Show Help"}
    </button>

    {showHelp && (
      <div className="help">
        Enter your message and submit it.
      </div>
    )}

  </div>
);
```

```
}

export default Contact;
```



GitHub Link:

https://github.com/hetavisuhagiya/AWDF-24DCE140